

BASE DE DATOS AVANZADAS Y BIG DATA

UPN.EDU.PE

Semana 12



Spark en Cluster y Operaciones sobre RDDS
(ResilientDistributed DataSets)

Presentación de la sesión

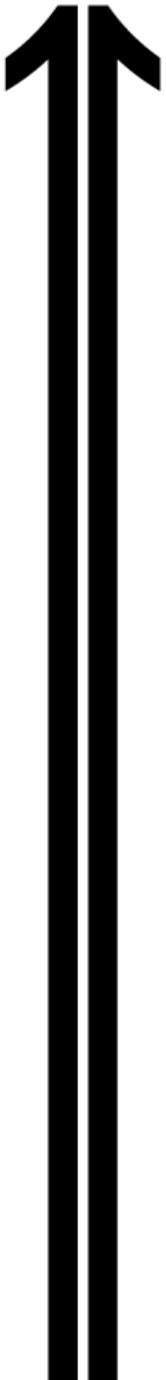


Logro de sesión

Al término de la sesión los estudiantes realizan operaciones sobre RDDs, comprendiendo como estos trabajan sobre un cluster de Spark.

Temario

- Spark en Cluster y Operaciones sobre RDDS (Resilient Distributed DataSets).



SPARK RDDS

APACHE SPARK

Simplicidad

Contar palabras en Hadoop

```java
package org.myorg;

import java.io.IOException;
import java.util.*;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

public class WordCount {

 public static class Map extends Mapper<LongWritable, Text, Text, IntWritable
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(LongWritable key, Text value, Context context) throws IOE
        String line = value.toString();
        StringTokenizer tokenizer = new StringTokenizer(line);
        while (tokenizer.hasMoreTokens()) {
            word.set(tokenizer.nextToken());
            context.write(word, one);
        }
    }
 }

 public static class Reduce extends Reducer<Text, IntWritable, Text, IntWrita

    public void reduce(Text key, Iterable<IntWritable> values, Context contex
      throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
 }

 public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();

        Job job = new Job(conf, "wordcount");

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);

    job.setInputFormatClass(TextInputFormat.class);
    job.setOutputFormatClass(TextOutputFormat.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    job.waitForCompletion(true);
 }

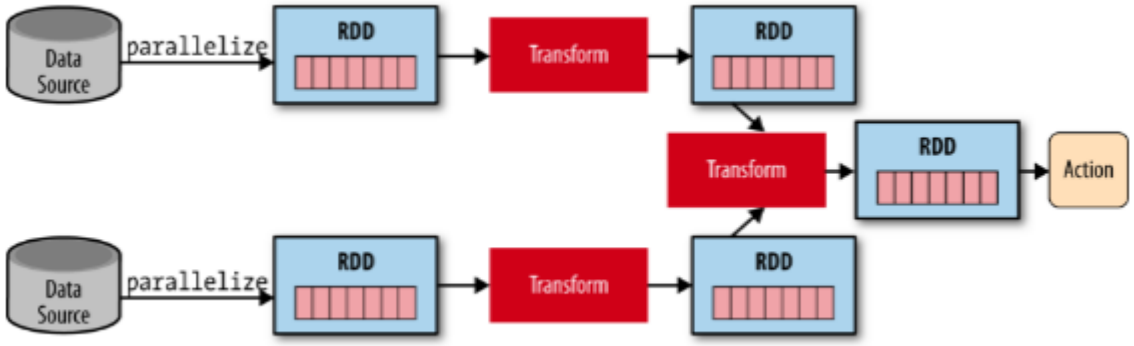
}
```

Contar palabras en Spark (python)

```python
text_file = sc.textFile("hdfs://...")
counts = text_file.flatMap(lambda line: line.split(" ")) \
             .map(lambda word: (word, 1)) \
             .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("hdfs://...")
```

SPARK - RDD

Los conjuntos de datos distribuidos resilientes (RDD) son colecciones de elementos tolerantes a fallas que se pueden distribuir entre varios nodos en un clúster y trabajar en paralelo. Los RDD son una estructura fundamental en Apache Spark



https://www.ibm.com/mx-es/topics/apache-spark#:~:text=Los%20conjuntos%20de%20datos%20distribuidos,estructura%20fundamental%20en%20Apache%20Spark.

RDDS



Resilient Distributed Datasets (RDDs)

- Trabajaremos sobre colecciones de datos denominadas RDD:
  - Es el concepto básico de trabajo en Spark
  - Son inmutables. Es decir una vez creados no se pueden modificar.
  - Se pueden transformar para crear nuevos RDDs o realizar acciones sobre ellos pero no modificar.
  - Se guarda la secuencia de transformaciones para poder recuperar RDDs de forma eficiente si alguna máquina se cae
  - Están distribuidos en el clúster en los nodos workers

CICLO DE VIDA DE UNA APLICACIÓN



RDD (Datos)

- Datos distribuidos en los workers
- Inmutables

Serie de transformaciones

- Operaciones cuyo resultado es otro RDD
- No se ejecutan inmediatamente

cache

- Algún RDD se puede mantener en memoria mediante la función cache()
- Evita recalcular

Serie de acciones

- Operaciones que devuelven resultados al driver
- Desencadenan la ejecución de las transformaciones definidas

FUNCIÓN LAMBDA



Recordatorio funciones lambda de python

- Son funciones anónimas. Por ejemplo, para sumar dos números:

```
lambda a, b: a + b
```

- Se pueden usar cuando haya que pasar una función como parámetro
- Tienen una única instrucción cuyo valor corresponde al valor devuelto

CREACIÓN DE UN RDD

Creación de RDD - parallelize

- Crea un RDD a partir de una lista python

```
numeros = sc.parallelize([1,2,3,4,5,6,7,8,9,10], 2)
```

Lista de python. Puede ser de números, cadenas…

Número de particiones en que se dividirá la lista (opcional)

- Evaluación perezosa

```
#Considerar que em Colab usaremos:
numeros = spark.sparkContext.parallelize([1, 1, 2, 2, 5],2)
```

CREACIÓN DE UN RDD



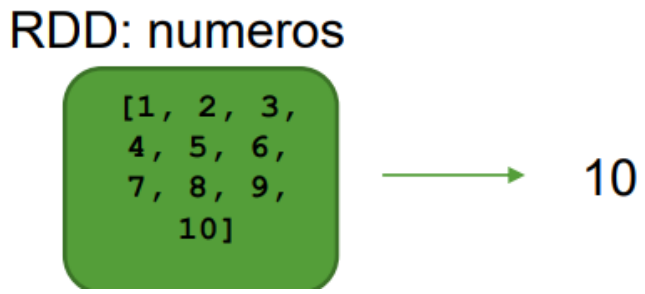
Creación de RDD

- Ejemplo evaluación perezosa

```
numeros = sc.parallelize([1,2,3,4,5,6,7,8,9,10])


print numeros.count()
```

- Spark “apunta” qué va a pasar
- No se calcula nada hasta que es necesario



RDD TRANSFORMACIONES



Transformaciones

- Crean un RDD a partir de otro u otros RDDs
- Evaluación perezosa. No se calculan los resultados inmediatamente. Spark apunta la serie de transformaciones que se deben aplicar para ejecutar después.
- Es como una receta

```
lineas.flatMap(...).filter(...).map(...).reduceByKey(...)
```

RDD TRANSFORMACIONES



Transformaciones generales

Transformación	Descripción
map(func)	Crea un nuevo RDD a partir de otro aplicando una transformación a cada elemento original
filter(func)	Crea un nuevo RDD a partir de otro manteniendo solo los elementos de la lista original que cumplan una condición
flatMap(func)	Como map pero cada elemento original se puede mapear a 0 o varios elementos de salida
distinct()	Crea un nuevo RDD a partir de otro eliminando duplicados
union(otroRDD)	Une dos RDD en uno
sample()	Obtiene un RDD con una muestra obtenida con reemplazamiento (o sin) a partir de otro RDD.

RDD TRANSFORMACIONES



Transformación - map

- Aplica una transformación a cada elemento del RDD original

```
numeros = sc.parallelize([1,2,3,4,5])


num3 = numeros.map(lambda elemento: 3*elemento)
```

Función que se aplica a cada elemento del rdd números

- Resultado: [1,2,3,4,5] → [3,6,9,12,15]
- La función que se pasa a map debe:
  - Recibir un único parámetro, que serán elementos individuales del rdd de partida
  - Devolver el elemento transformado

RDD TRANSFORMACIONES



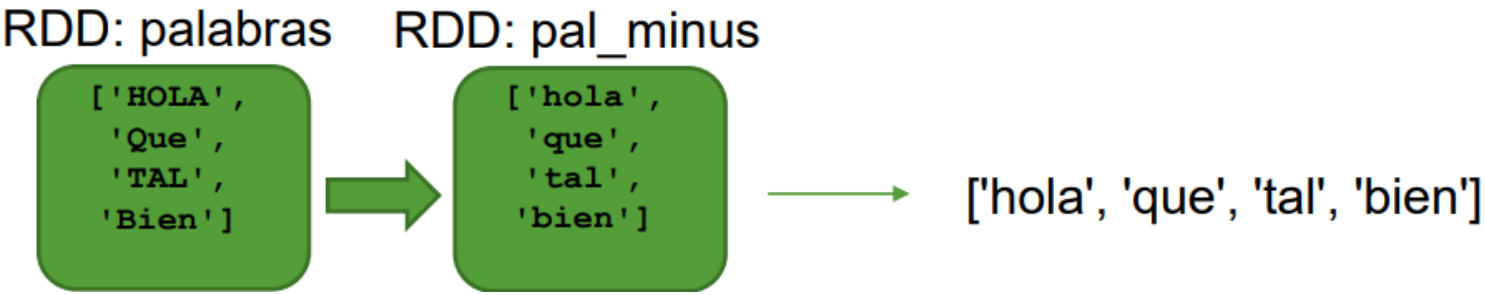
Transformación – cuestiones sobre el map

- ¿Cuál es el tamaño del rdd de salida?
  - El mismo que el tamaño de entrada

```
palabras = sc.parallelize(['HOLA', 'Que', 'TAL', 'Bien'])

pal_minus = palabras.map(lambda elemento: elemento.lower())

print pal_minus.collect()
```



RDD TRANSFORMACIONES



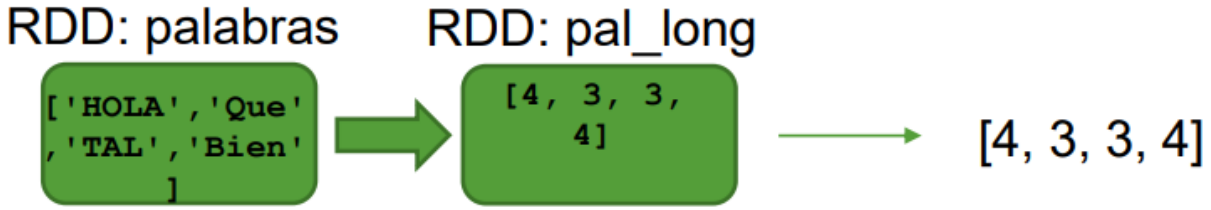
Transformación – cuestiones sobre el map

- ¿Podemos cambiar el tipo de los elementos de los RDDs con un map?
  - No, los RDDs son inmutables!! Pero con map podemos crear nuevos RDDs

```
palabras = sc.parallelize(['HOLA', 'Que', 'TAL', 'Bien'])


pal_long = palabras.map(lambda elemento: len(elemento))


print pal_long.collect()
```



RDD TRANSFORMACIONES



Transformación - filter

- Filtra un RDD manteniendo solo los elementos que cumplan una condición

```
numeros = sc.parallelize([1,2,3,4,5])


rdd = numeros.filter(lambda elemento: elemento%2==0)
```

Función que se aplica a cada elemento para filtrarlo

- Resultado: [1,2,3,4,5] → [2,4]
- La función que se pasa a filter debe:
  - Recibir un único parámetro, que serán elementos individuales del rdd de partida
  - Devolver True o False para indicar si el elemento pasa o no el filtro

RDD TRANSFORMACIONES



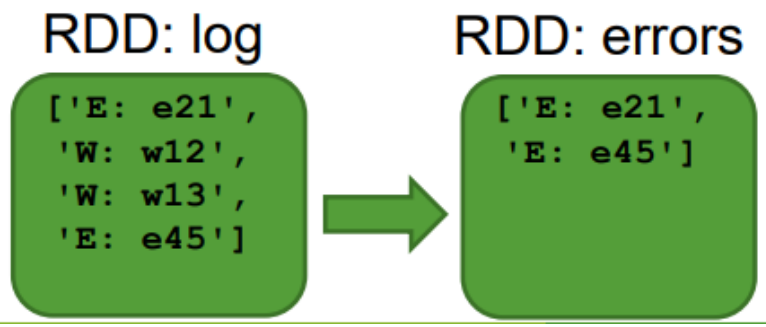
Transformación – cuestiones sobre el filter

- ¿Cuál es el tamaño del rdd de salida?
  - Menor o igual que el original

```
log = sc.parallelize(['E: e21', 'W: w12', 'W: w13', 'E: e45'])


errors = log.filter(lambda elemento: elemento[0]=='E')


print errors.collect()
```



El RDD de salida es del mismo tipo que el de entrada

['E: e21', 'E: e45']

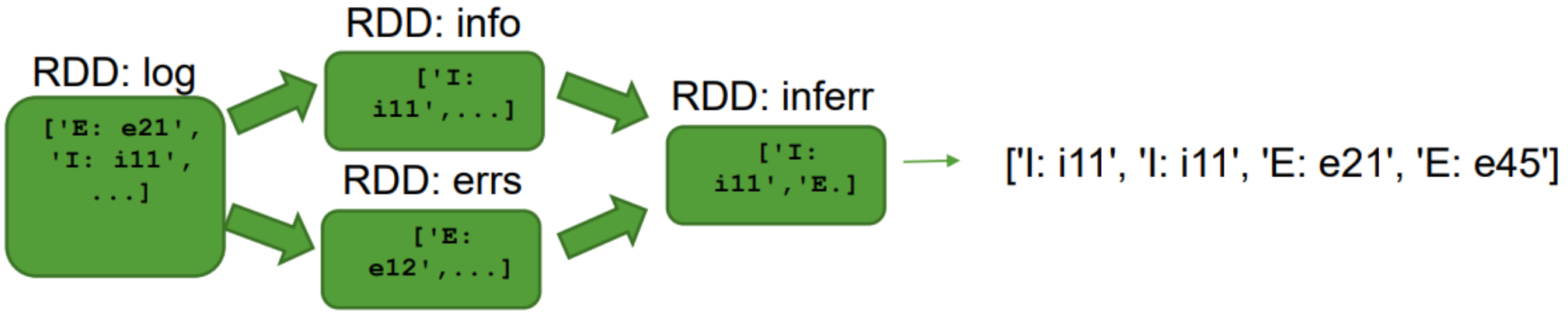
RDD TRANSFORMACIONES

Transformación – union otro ejemplo

```
log = sc.parallelize(['E: e21', 'I: i11', 'W: w12', 'I: i11', 'W: w13', 'E: e45'])

info = log.filter(lambda elemento: elemento[0]=='I')
errs = log.filter(lambda elemento: elemento[0]=='E')
inferr = info.union(errs)

print inferr.collect()
```



RDD ACCIONES



Acciones básicas

Acción	Descripción
count()	Devuelve el número de elementos del RDD
reduce(func)	Agrega los elementos del RDD usando *func*
take(n)	Devuelve una lista con los primeros n elementos del RDD
collect()	Devuelve una lista con todos los elementos del RDD
takeOrdered(n[,key=func])	Devuelve n elementos en orden ascendente. Opcionalmente se puede especificar la clave de ordenación

RDD ACCIONES



Acción - reduce

- Agrega todos los elementos del RDD por pares hasta obtener un único valor

```
numeros = sc.parallelize([1,2,3,4,5])


print numeros.reduce(lambda elem1,elem2: elem1+elem2)
```

- Resultado: → 15
- La función que se pasa a reduce debe:
  - Recibir dos argumentos y devolver uno **de tipo compatible**
  - Ser conmutativa y asociativa de forma que se pueda calcular bien el paralelo

RDD ACCIONES



Acción - reduce

- Otro ejemplo

```
palabras = sc.parallelize(['HOLA', 'Que', 'TAL', 'Bien'])


pal_minus = palabras.map(lambda elemento: elemento.lower())


print palabras.reduce(lambda elem1,elem2: elem1+ "-" + elem2)
```

- Resultado: “hola-que-tal-bien”
- ¿Tiene sentido esta operación?
  - No del todo. Aquí ha salido bien pero no es conmutativa
  - **¿Qué pasa si ponemos elem2+ "-" + elem1?**

RDD ACCIONES



Acción - take

- Devuelve una lista con los primeros n elementos del RDD

```
numeros = sc.parallelize([5,3,2,1,4])


print numeros.take(3)
```

- Resultado: → [5,3,2]

RDD ACCIONES



Acción - collect

- Devuelve una lista con todos los elementos del RDD

```
numeros = sc.parallelize([5,3,2,1,4])


print numeros.collect()
```

- Resultado: → [5, 3, 2, 1, 4]
- Cuando se llama a collect todos los datos del RDD se envían al driver program

¡¡Hay que estar seguros que caben en memoria!!

RDD ACCIONES



Acción - takeOrdered

- Devuelve una lista con los primeros n elementos del RDD en orden

```
numeros = sc.parallelize([3,2,1,4,5])


print numeros.takeOrdered(3)
```

- Resultado: → [1,2,3]

¿Preguntas o comentarios?



CONCLUSIONES

RDD es inmutable y tolerante a fallos que se pueden distribuir entre varios nodos en un clúster y trabajar en paralelo. Los RDD son una estructura fundamental en Apache Spark

Cada conjunto de datos en un RDD se divide en particiones lógicas, que pueden calcularse en diferentes nodos del clúster. Y los usuarios pueden realizar dos tipos de operaciones RDD: transformaciones y acciones

Las **Transformaciones en los RDDs** no se ejecutan inmediatamente, a esto se le conoce como evaluación perezosa



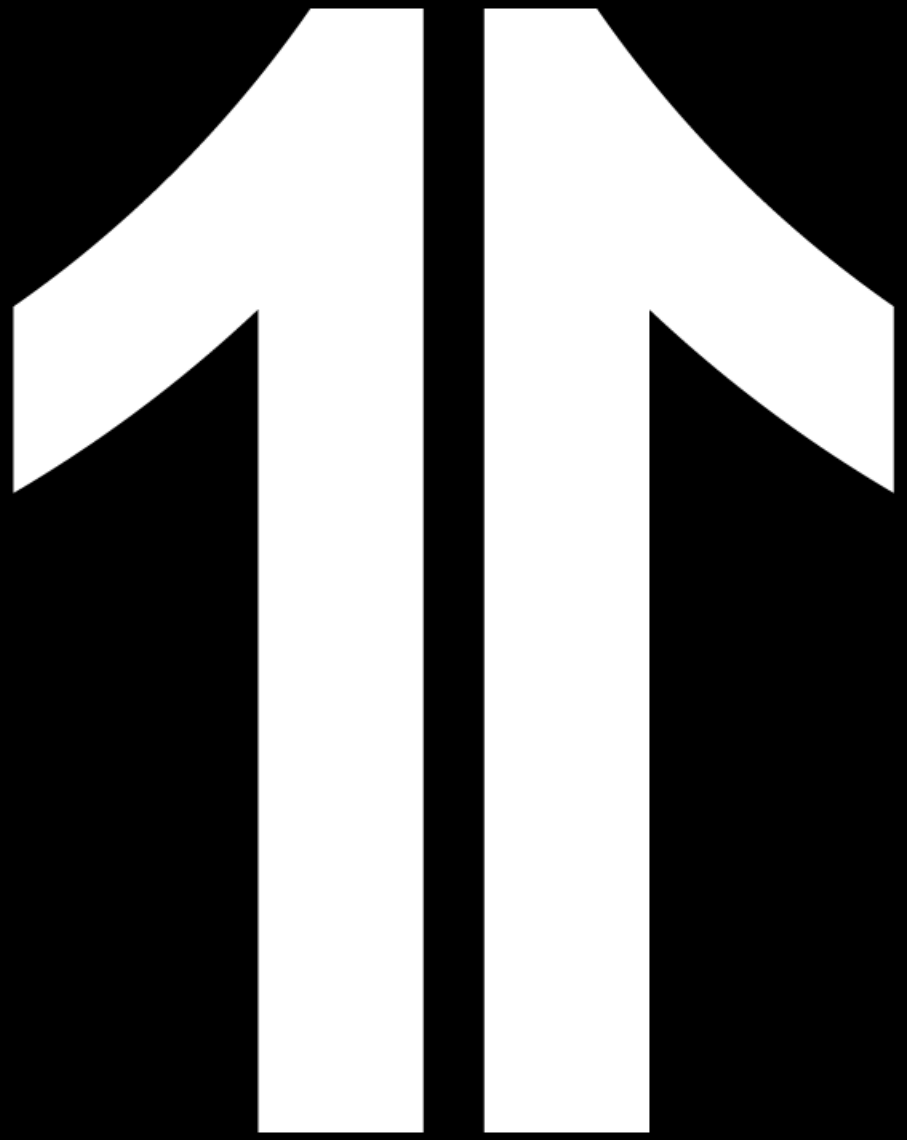
REFERENCIAS BIBLIOGRÁFICAS

Qué es Apache spark?

https://www.ibm.com/mx-es/topics/apache-spark#:~:text=Los%20conjuntos%20de%20datos%20distribuidos,estructura%20fundamental%20en%20Apache%20Spark.

Qué es RDD?
https://keepcoding.io/blog/rdd-resilient-distributed-datasets/

Video – Ejemplo RDD
https://www.youtube.com/watch?v=deI3qVG7jbk



GRACIAS


UPN
UNIVERSIDAD
PRIVADA
DEL NORTE